

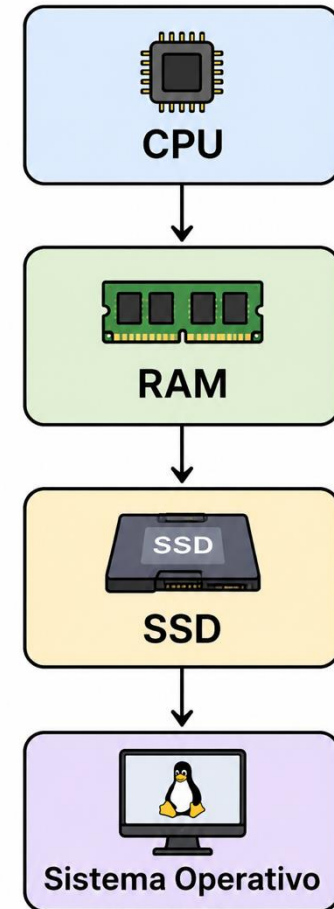
A 3D rendering of a computer circuit board, viewed from an isometric perspective. The board is dark grey with a grid of gold-colored traces. A central square integrated circuit (CPU) is prominent, surrounded by several smaller square components, likely memory modules or cache chips. A larger, rectangular component with a grid of pins is visible on the right side. The overall aesthetic is technical and modern.

Administración de Memoria y Almacenamiento

Sistemas Operativos 1151018

Introducción

- Revisar la administración de memoria.
- Comprender el funcionamiento de la memoria virtual.
- Describir el funcionamiento del DMA.
- Revisar la organización de un sistema de archivos.
- Integrar todos los componentes del sistema operativo.

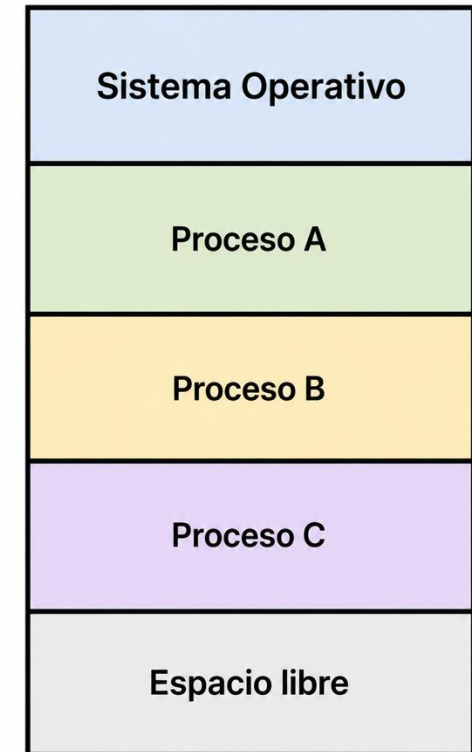


¿Por qué necesitamos administrar la memoria?

El CPU ejecuta procesos desde la memoria RAM, nunca directamente desde el disco.

En memoria se almacenan los procesos y cada proceso tiene:

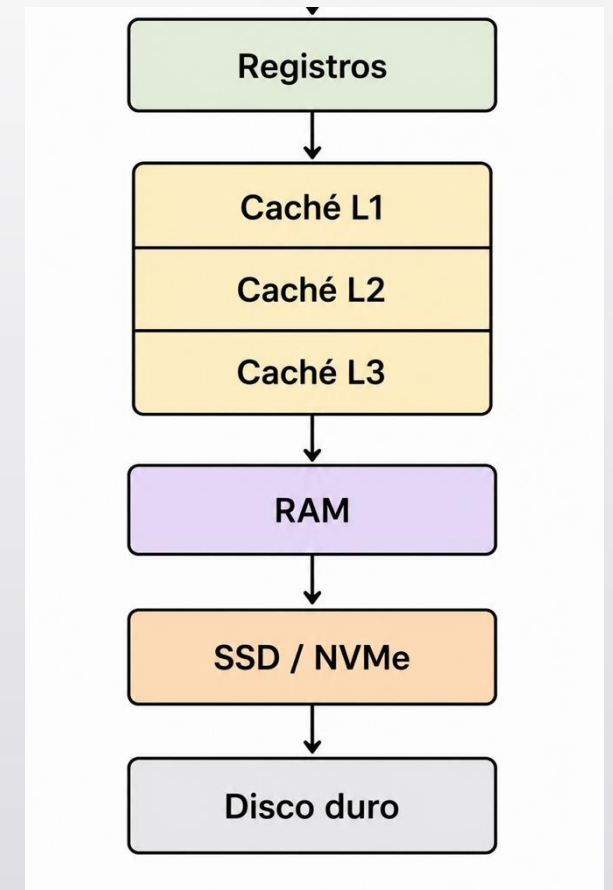
- Código del programa.
- Variables.
- Funciones.
- Bibliotecas.
- Pila (Stack).
- Datos dinámicos (Heap).



Funciones del Administrador de Memoria

El administrador de memoria tiene cuatro funciones principales:

1. **Asignar memoria.** Reservar espacio para nuevos procesos.
2. **Proteger memoria.** Evitar que un proceso modifique la memoria de otro.
3. **Compartir memoria.** Permitir compartir información cuando sea necesario.
4. **Liberar memoria.** Recuperar la memoria cuando un proceso termina.



Funciones del Administrador de Memoria

¿Qué ocurriría si un programa nunca libera la memoria que utilizó?

Si todos los procesos necesitan memoria, ¿por qué una computadora tiene diferentes tipos: registros, caché, RAM, SSD y disco duro?

¿No bastaría con un solo tipo de memoria?



La jerarquía de memoria

- La memoria de una computadora está organizada en niveles.
- Esta ordenada según:
 - ✓ Velocidad,
 - ✓ Capacidad,
 - ✓ Costo por byte,
 - ✓ Proximidad al procesador.



¿Por qué el CPU no ejecuta programas directamente desde el SSD?

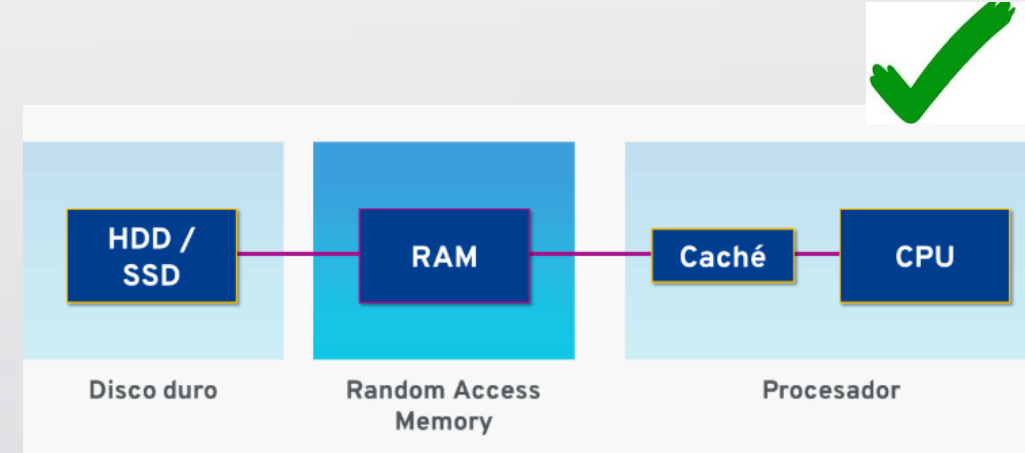
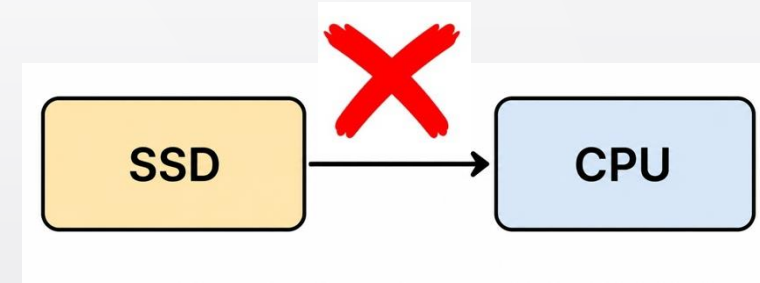
Comparación de tiempos de acceso

Tipo de memoria	Tiempo aproximado
Registros	~0.3 ns
Caché L1	~1 ns
Caché L2	~4 ns
Caché L3	~10 ns
RAM	~80 ns
SSD NVMe	~20–100 µs
Disco duro (HDD)	~5–10 ms

1 microsegundo equivale a 1,000 nanosegundos.
1 milisegundo equivale a 1,000,000 de nanosegundos.

¿Por qué el CPU no ejecuta programas directamente desde el SSD?

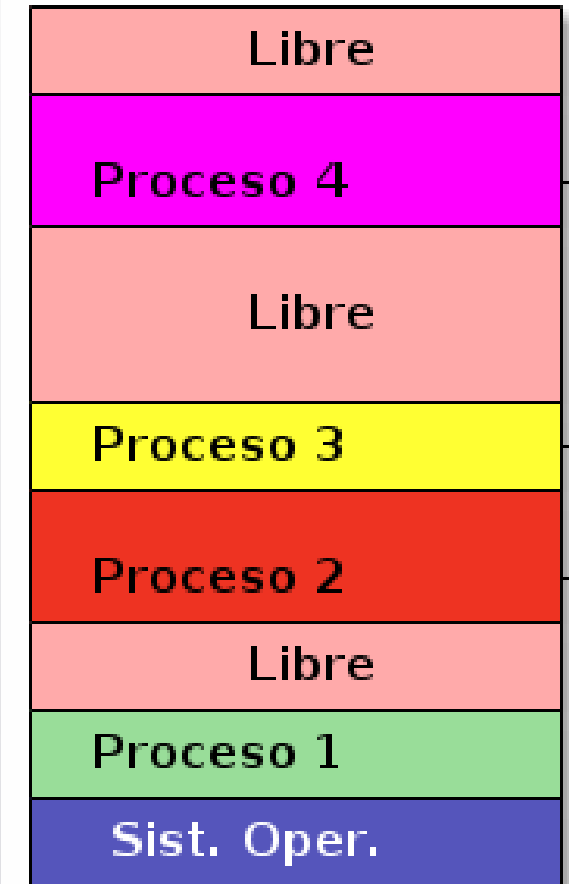
- Acceder a un disco duro puede ser **cientos de miles o incluso millones de veces más lento** que acceder a la memoria RAM.
- Por eso el procesador **nunca ejecuta directamente desde el disco**.
- Primero el sistema operativo copia el programa a la RAM y, desde ahí, el CPU comienza a ejecutar las instrucciones.
- Éste es el motivo por el que la memoria RAM sigue siendo indispensable, incluso cuando los SSD modernos son muy rápidos.



Organización de la Memoria RAM

- La memoria RAM se comparte entre el sistema operativo y los procesos en ejecución.
- Características
 - ✓ La RAM tiene una capacidad limitada.
 - ✓ Cada proceso ocupa una región de memoria diferente.
 - ✓ El sistema operativo administra toda la memoria disponible.
 - ✓ Cuando un proceso termina, su memoria puede reutilizarse.

¿Qué sucede cuando abrimos un programa y ya no existe suficiente espacio libre en la memoria RAM?



Protección de la Memoria RAM

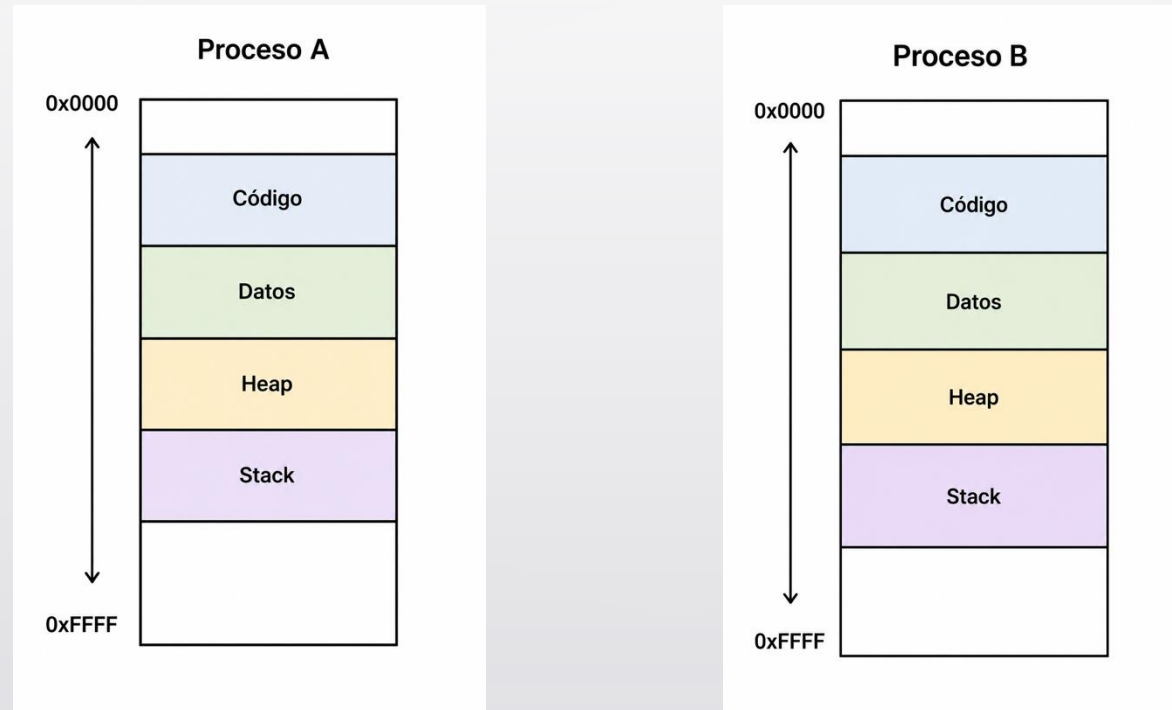
- Existen varios procesos, cada uno tiene su propia memoria,
- Ninguno puede invadir al otro para evitar:
 - ✓ Corrupción de datos
 - ✓ Fallos en las aplicaciones
 - ✓ Bloqueo del sistema.

```
int arreglo[10];  
arreglo[100] = 5;
```

¿Cómo sabe el sistema operativo qué parte de la memoria pertenece a cada proceso?

Espacio de Direcciones

- Cada proceso posee su propio espacio de direcciones.

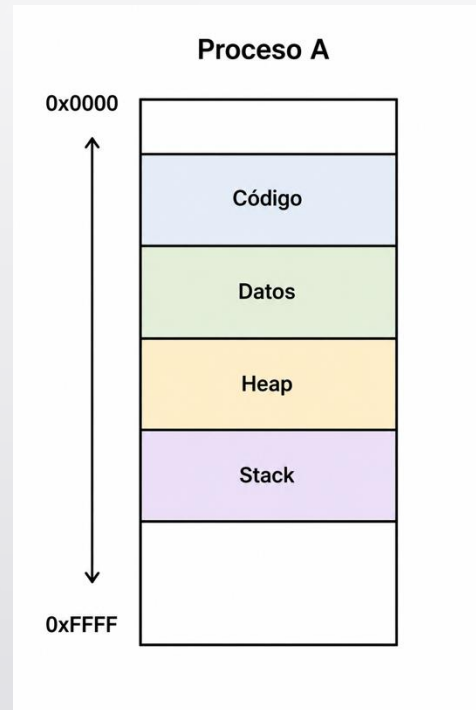


Cada proceso cree que dispone de un espacio completo de memoria, independiente de los demás procesos.

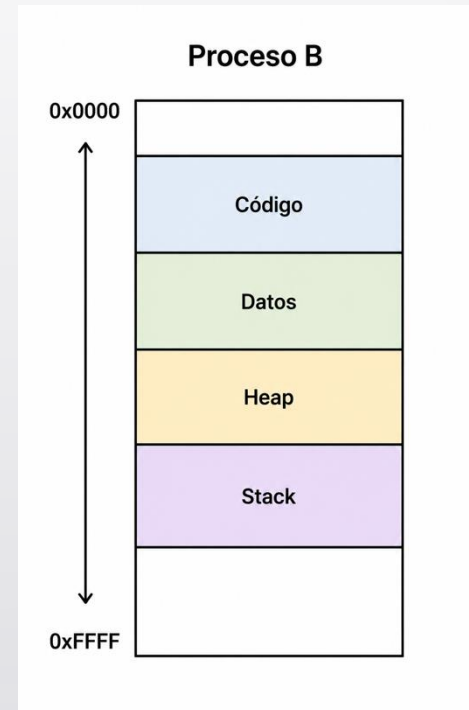
Espacio de Direcciones

- Cada proceso posee su propio espacio de direcciones.

```
// Programa A  
int contador = 10;
```



```
// Programa B  
int contador = 50;
```



¿Cómo es posible que ambos utilicen exactamente las mismas direcciones sin interferirse?

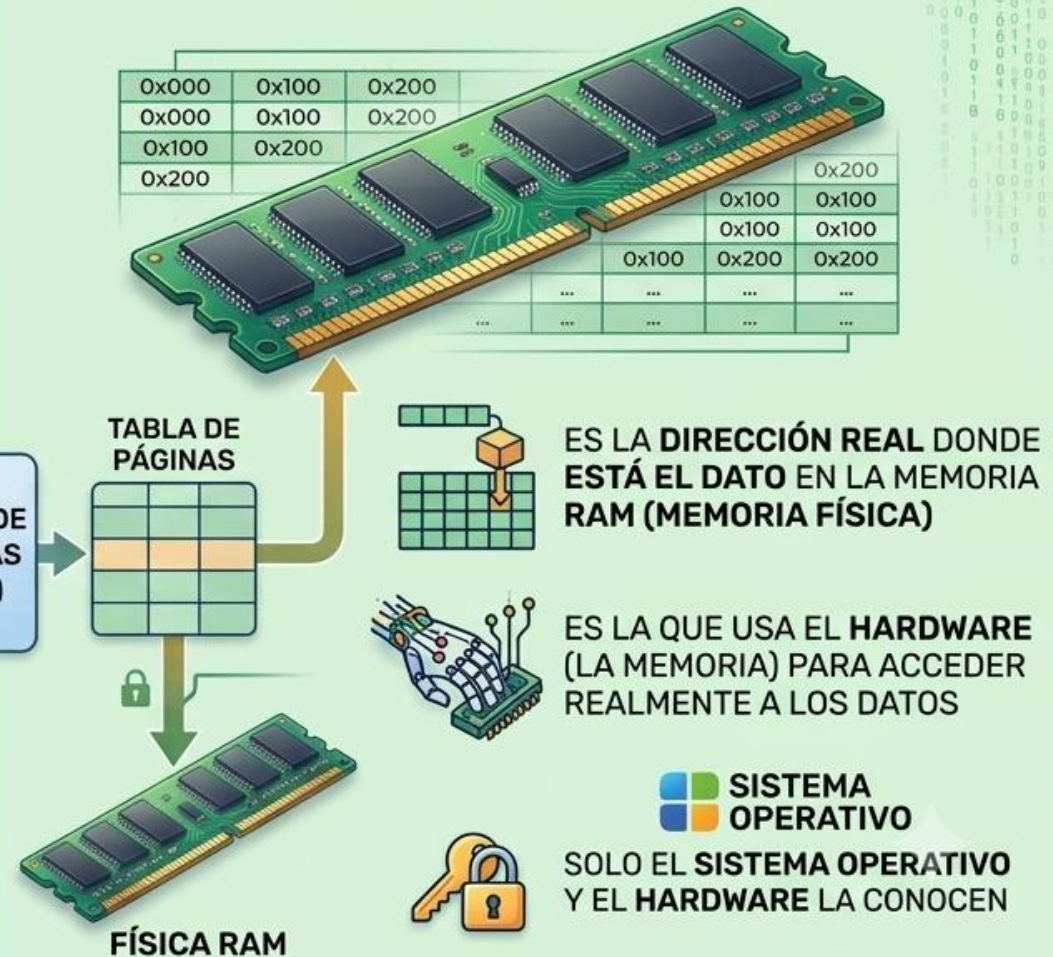
Cada proceso cree que dispone de un espacio completo de memoria, independiente de los demás procesos.

DIRECCIONAMIENTO DE MEMORIA: LÓGICA VS. FÍSICA

1. DIRECCIÓN LÓGICA (O VIRTUAL)

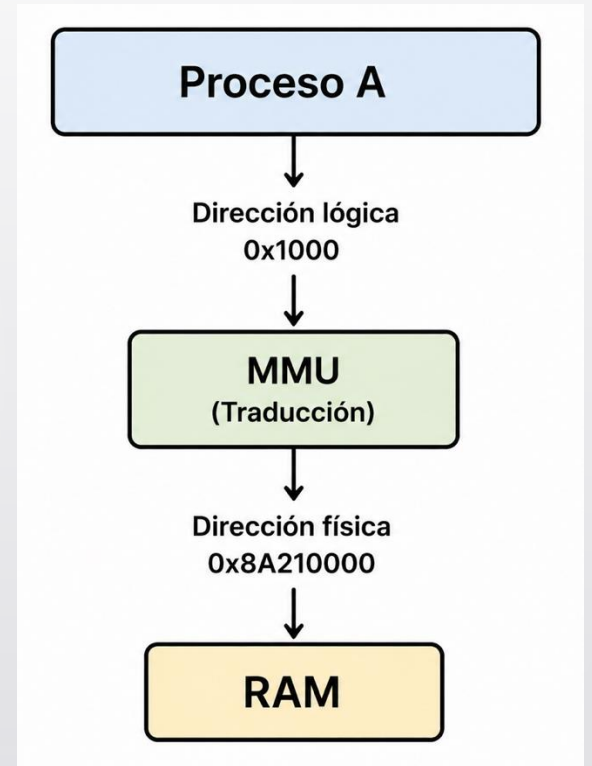


2. DIRECCIÓN FÍSICA



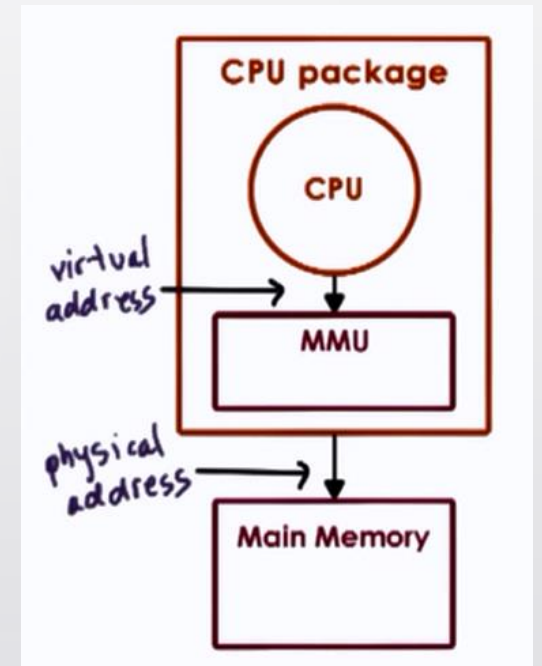
Unidad de Gestión de Memoria (MMU)

- Cuando un programa se ejecuta, no accede directamente a la memoria física. En su lugar, el sistema operativo le hace creer que tiene todo un espacio de direcciones para él solo.
- La MMU es la pieza que hace posible esa ilusión, **traduciendo cada dirección virtual a su dirección física correspondiente** en tiempo real, cada vez que la CPU accede a memoria



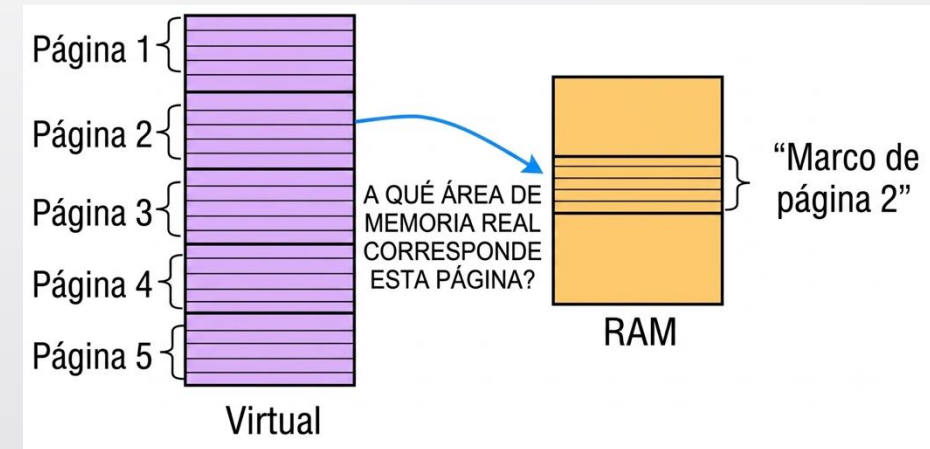
Funciones principales de la MMU

- **Traducción de direcciones:** convierte direcciones virtuales en direcciones físicas usando estructuras como las **tablas de páginas**.
- **Protección de memoria:** evita que un proceso acceda a la memoria de otro proceso o a zonas restringidas del sistema (por ejemplo, memoria del kernel).

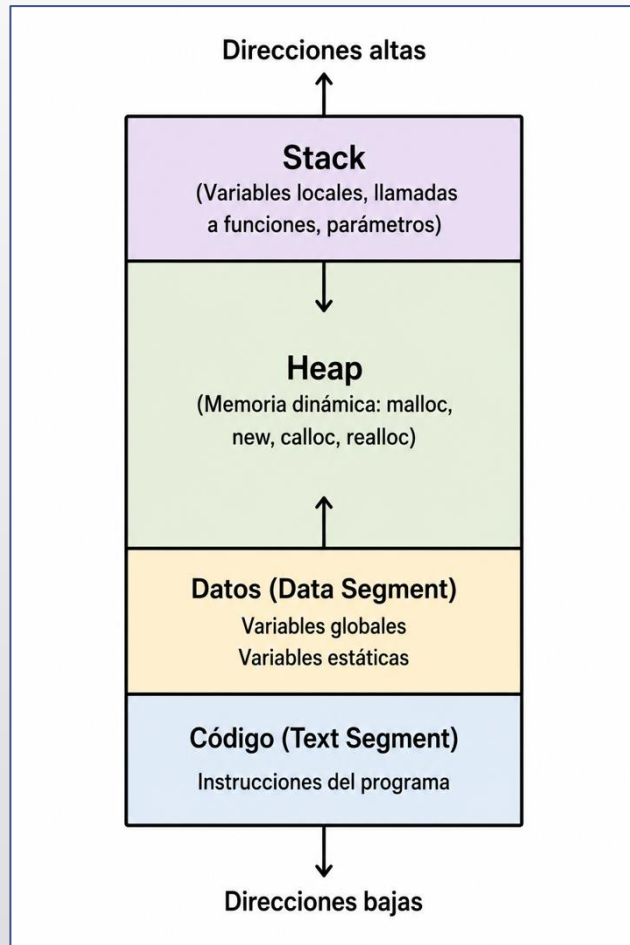


Funciones principales de la MMU

- **Soporte para memoria virtual:** permite que el sistema operativo implemente **paginación**, donde la memoria se divide en bloques fijos llamados **páginas** (bloque de tamaño fijo del espacio de direcciones lógicas de un proceso, tamaño de típico 4KB) y **marcos** (bloque de tamaño fijo de la memoria física). La RAM completa se divide en marcos del mismo tamaño que las páginas.



Organización de un Proceso en Memoria



```
#include <stdio.h>
#include <stdlib.h>

int contador = 100;

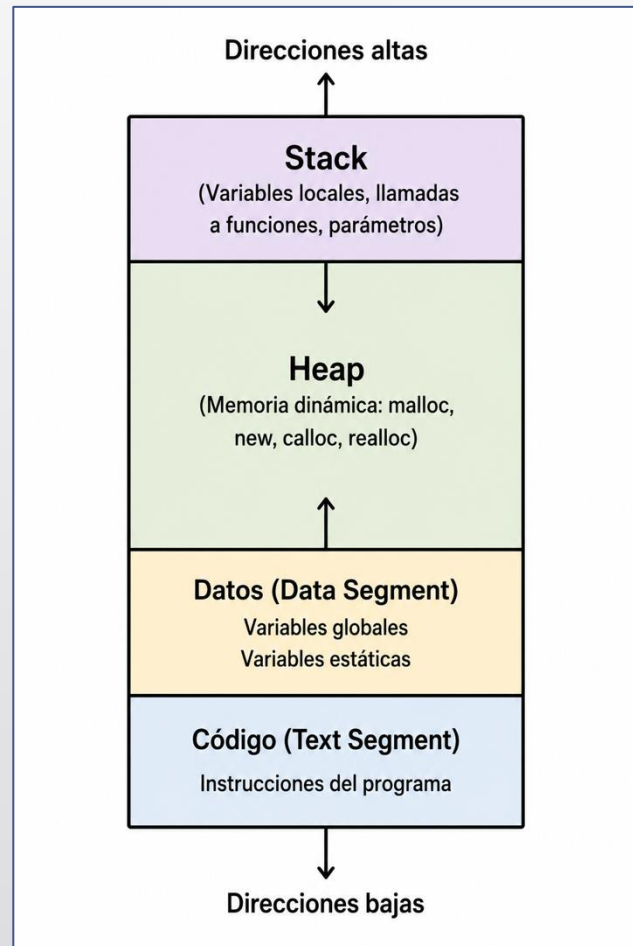
int main() {

    int edad = 20;

    int *vector = (int *)malloc(100*sizeof(int));

    return 0;
}
```


Organización de un Proceso en Memoria



```
#include <stdio.h>
#include <stdlib.h>

int contador = 100;

int main() {

    int edad = 20;

    int *vector = (int *)malloc(100*sizeof(int));

    return 0;
}
```

¿Qué ocurre si el programa necesita más memoria de la que realmente existe en la RAM?



Memoria Virtual

Problema

Si la computadora tiene **8 GB de RAM** y el usuario ejecuta:

- Google Chrome
- Visual Studio
- MATLAB
- Máquina Virtual
- Spotify

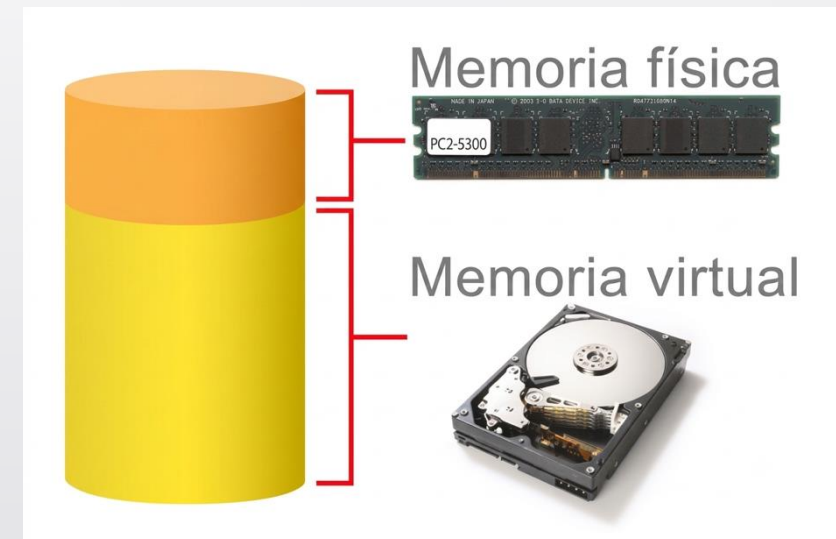
Memoria requerida = 12 GB



¿Cómo puede ejecutarse si la RAM sólo tiene 8 GB?

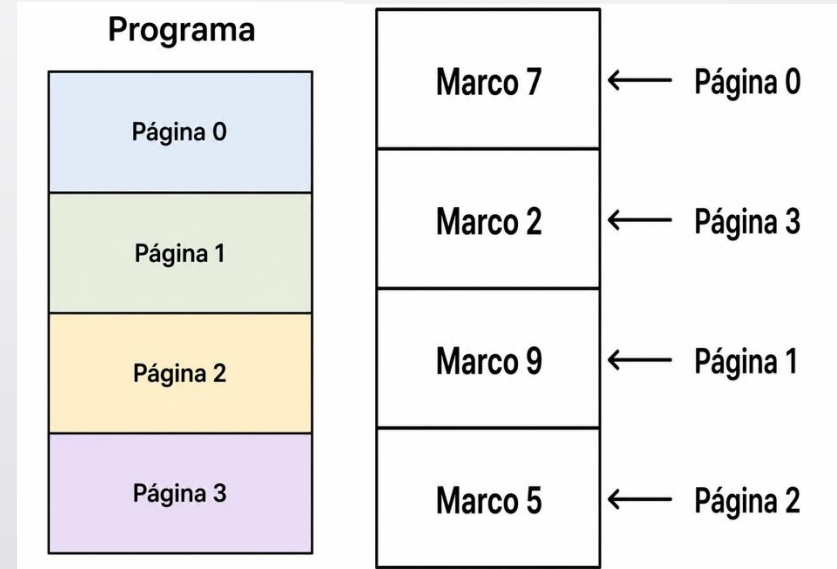
Memoria Virtual

- Es una técnica del SO que utiliza a la memoria secundaria como respaldo cuando la RAM no es suficiente.
- Las partes que se están utilizando permanecen en la memoria principal.
- Las partes que no se utilizan temporalmente permanecen en el almacenamiento secundario (SSD o disco).
- De esta forma, el proceso tiene la impresión de disponer de una memoria mucho más grande que la memoria física instalada.



¿Cómo funciona?

- El espacio lógico se divide en bloques de tamaño fijo llamados **páginas**, y la memoria física (RAM) se divide en bloques del mismo tamaño llamados **marcos** (normalmente de 4 KB).
- El sistema operativo mantiene una **tabla de páginas** (page table) por cada proceso. Esta tabla traduce cada página virtual al marco físico donde está almacenada.



¿Cómo funciona?

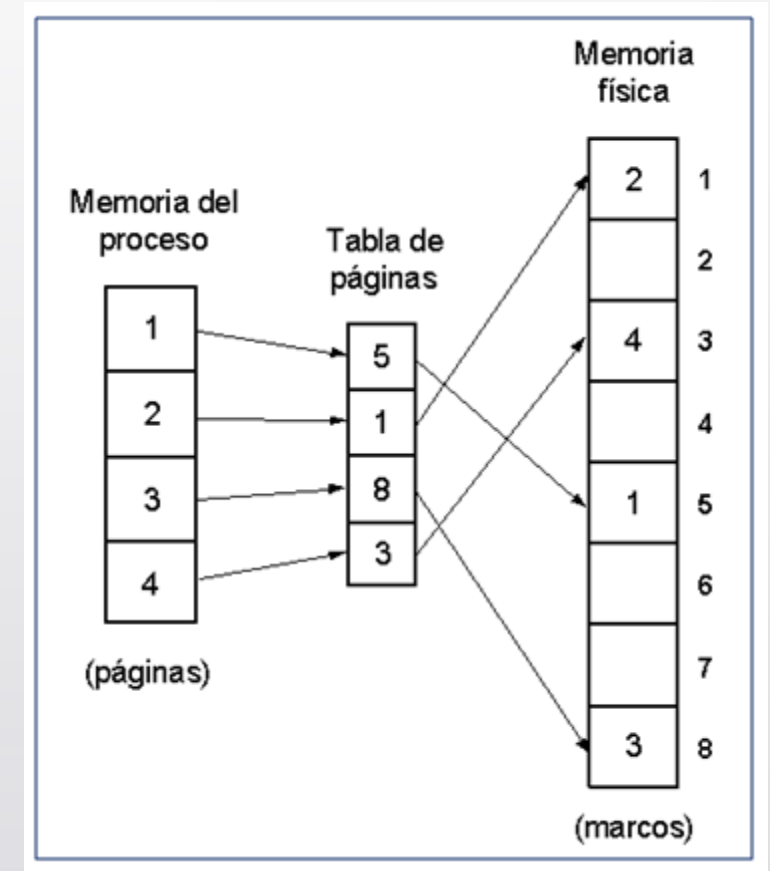
- Cuando un programa accede a una dirección virtual:
 - El hardware (**MMU**) consulta la tabla de páginas.
 - Si la página está cargada en algún marco de la RAM → accede normalmente
 - Si la página no está en RAM (fallo de página) → el sistema operativo hará lo siguiente:



¿Cómo funciona?

1. Busca la página en el disco (archivo de paginación o swap).
2. La carga en un marco libre de la RAM (posiblemente sacando de la RAM otra página menos usada).
3. Actualiza la tabla de páginas.
4. Reanuda la ejecución del programa.

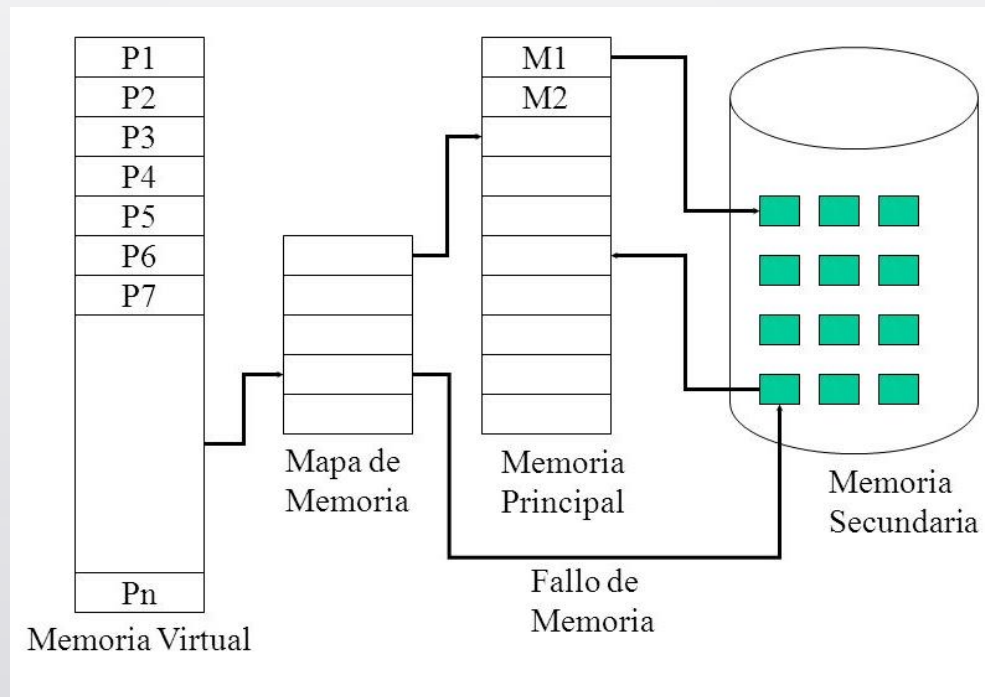
Nota: Si el sistema pasa más tiempo moviendo páginas entre el SSD y la RAM que ejecutando los programas, se produce un fenómeno llamado **Thrashing**.



Memoria Virtual

¿Qué pasa si ocurre un fallo de página y todos los marcos de la RAM están ocupados.?

El sistema tiene que decidir **qué página sacar** para hacerle espacio a la nueva.





Algoritmos de reemplazo de páginas

1. FIFO (First In, First Out)

- ✓ La primera página que entró en el marco es la primera que sale.
- ✓ Fácil de implementar
- ✓ **Problema:** puede sacar una página muy usada solo porque "ya lleva rato ahí", lo cual es ineficiente.

FIFO (First In, First Out)

cadena de referencia

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2														7	7	7
	0	0	0														1	0	0
		1	1														2	2	1

marcos de página



Algoritmos de reemplazo de páginas

2. Óptimo (OPT / MIN)

Saca la página que **no se va a usar durante más tiempo en el futuro**.

- ✓ Es el algoritmo *ideal*, produce la menor cantidad posible de fallos de página.
- ✓ Se usa solo como punto de referencia teórico
- ✓ **Problema:** requiere saber el futuro (qué páginas se van a pedir después), lo cual es imposible en un sistema real

Óptimo (OPT / MIN)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames



Algoritmos de reemplazo de páginas

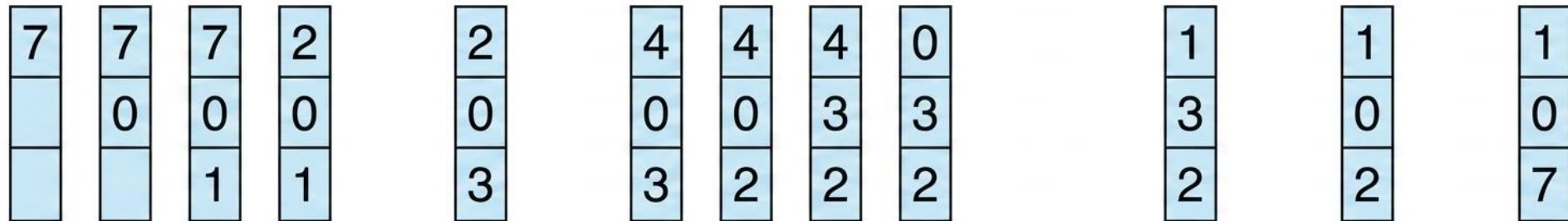
3. LRU (Least Recently Used)

- ✓ Saca la página que lleva más tiempo sin usarse. Este algoritmo utiliza información pasada en lugar de información futura.
- ✓ Si algo no se ha usado en un rato, probablemente no se use pronto.
- ✓ Más "inteligente" que FIFO porque sí toma en cuenta el comportamiento real del programa.
- ✓ **Problema:** es costoso de implementar con precisión perfecta (hay que llevar registro de cuándo se usó cada página cada vez)

LRU (Least Recently Used)

cadena de referencia

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



marcos de página



Algoritmos de reemplazo de páginas

4. LFU (menor frecuencia de acceso)

Saca la página que se ha usado menos veces en total, sin importar cuándo fue la última vez.

- ✓ Mantiene un contador de accesos por página.
- ✓ **Problema:** una página puede haber sido muy usada en el pasado y ya no necesitarse, pero como tiene un contador alto, tarda en ser eliminada. También hay que decidir qué hacer con páginas recién cargadas que aún tienen contador bajo.



Algoritmos de reemplazo de páginas

5. Reloj / Segunda oportunidad (Clock / Second Chance)

Es una aproximación práctica y eficiente de LRU, mucho más fácil de implementar.

- ✓ Las páginas se organizan en un círculo (como las horas de un reloj), cada una con un **bit de referencia** (0 o 1).
- ✓ Un puntero recorre el círculo. Si encuentra una página con bit = 1, le da "una segunda oportunidad": pone el bit en 0 y sigue avanzando (no la saca).
- ✓ Si encuentra una página con bit = 0, esa es la que se saca.
- ✓ Es el algoritmo que más se usa en sistemas reales porque es un buen balance entre eficiencia y buen rendimiento

Durante un **fallo de página**, el sistema operativo debe copiar una página desde el SSD hasta la memoria RAM.

¿Quién realiza físicamente esa copia?

¿El procesador mueve los datos byte por byte?

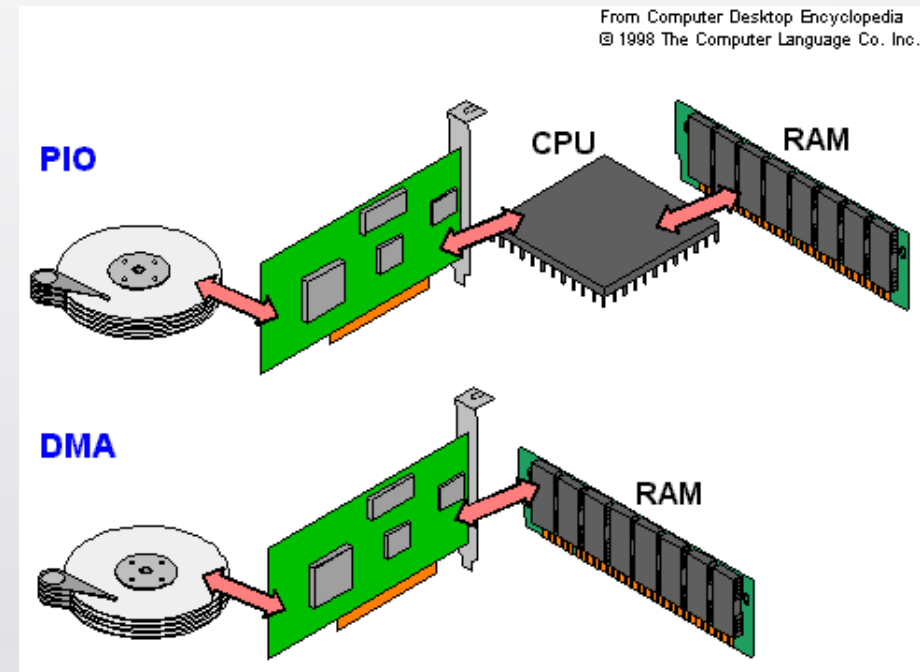




DMA (Acceso Directo a Memoria)

DMA (Direct Memory Access)

- Es un mecanismo que permite a los dispositivos de hardware (tarjetas de red, discos duros, tarjetas de sonido, GPUs, etc.) transferir datos directamente a la memoria RAM, sin necesidad de que la CPU intervenga en cada byte.
- **Sin DMA** (Programación I/O): La CPU debe leer/escribir cada byte → CPU muy ocupada.
- **Con DMA**: La CPU solo inicia y recibe la notificación final → mucho más eficiente.



Funcionamiento Básico

1. La CPU le indica al controlador DMA:

- Dirección de origen
- Dirección de destino en RAM
- Cantidad de datos a transferir

2. El controlador DMA toma el control del bus del sistema.

3. Transfiere los datos **directamente** entre el dispositivo y la memoria RAM.

4. Una vez finalizada la transferencia, el DMA genera una **interrupción** a la CPU para avisarle.

